a line by line search on the file for the pattern given by the compiled regular expression object called *pat*. The object *pat* describing the regular expression is compiled in the Python shell and not in the script, so that the same Python script *run.py* can be called to process different regular expressions without any modification to the script. The Python shell can be invoked by typing the command *python* in the terminal. The script *run.py* reads the file name from the keyboard; so different text files can be processed with this Python script.

```
filename = raw_input('Enter File Name to Process: ')
with open(filename) as fn:
     for line in fn:
       m = pat.search(line)
       if m:
           print m.group( )
```

Now it is time to understand how the script *run.py* works. The name of the file to be processed is read into a variable called the filename. The *with* statement of Python, introduced in Python 2.5, is used to open and close the required file. The file is then read line by line to find a match for the required regular expression with a *for* loop. The line of code, *m = pat.search(line)* searches for the pattern described by the regular expression in the compiled pattern object 'pat' in the string stored in the variable 'line'. It returns a 'Match' object if a match is found or a 'None' object if a match is not found. This returned object is saved in the object 'm' for further processing. The line of code 'if m:' checks whether the object 'm' contains a 'Match' object or a 'None' object. If object 'm' is 'None' then the *if* conditional fails and no action is taken. But on the other hand if 'm' contains a 'Match' object, then the matched string is printed on the screen by the line *print m.group( )*. The method *group( )* is defined for the object 'Match' and it returns the string matched by the regular expression.

Figure 2 shows the output obtained by the regular expression with and without the compiler flag IGNORECASE enabled. If you observe the figure carefully, you will see that the module *re* is imported first and then the pattern is compiled in the Python shell with the line of code, *pat = re.compile('UNIX')*. Then the line of code, *execfile(run.py)* executes the Python script *run.py* and the output of this case-sensitive search results in the match of a single string *UNIX*. As mentioned earlier, the function *compile( )* has many optional flags. The pattern object *pat* is recompiled a second time with the line of code, *pat = re.compile('UNIX',re.IGNORECASE)* executed on the Python shell with the flag IGNORECASE enabled. The script *run.py* is executed again and this case-insensitive search results in the match of strings *unix*, *Unix* and *UNIX*.

In the script *run.py*, if you replace the line of code *print m.group( )* with the code *print line*, the whole line in which a match is found will be printed. This Python script is called *line.py*. For example, for the pattern, *pat = re.compile('UNIX')* the modified script will print *UNIX IS AN OPERATING SYSTEM* instead of *UNIX*.

```
>>> import re
>>> pat = re.compile('UNIX')
>>> execfile('run.py')
Enter File Name to Process: file1.txt
UNIX
>>> pat = re.compile('UNIX',re.IGNORECASE)
>>> execfile('run.py')
Enter File Name to Process: file1.txt
unix
Unix
UNIX
```

Figure 2: Optional flags in the *compile( )* function

The method *group( )* is not the only method defined for the object match. The other methods defined are *start( ), end( ), span( )* and *groups( )*. The method *start( )* returns the starting position of the match and the method *end( )* returns the ending position of the match. The method *span( )* returns the starting and ending positions as a tuple. For example, if you replace the line of code *print m.group( )* in the script *run.py* with the code *print m.span( )* with the pattern *pat = re.compile('UNIX')* then the tuple (0,4) will be printed. This Python script is called *span.py*. In order to understand the working of *groups( )* method we need to understand the meaning of the special symbols used in Python regular expressions.

Special symbols in Python regular expressions

The following characters: . (dot), ^ (caret), $ (dollar), * (asterisk), + (plus), ? (question mark), { (opening curly bracket), } (closing curly bracket), [(opening square bracket),] (closing square bracket), \ (backslash), | (pipe), ((opening parenthesis) and) (closing parenthesis) are the special symbols used in Python regular expressions. They have special meaning and hence using them in a regular expression will not lead to a literal match for these characters.

The most important special symbol is backslash (\) which is used for two purposes. First, backslash can be used to create more meta characters in regular expressions. For example, *\d* means any decimal digit, *\D* means any non-decimal digit, *\s* means any whitespace character, *\S* means any non-whitespace character and *\n*, *\t*, etc, all have their usual meaning. Second, if a special symbol is prefixed with a backslash, then its special meaning is removed and thereby results in the literal match of that special symbol. For example, ** matches a ** and *\$* matches a *$*. The backslash creates some problems because it is a special symbol in Python regular expressions as well as Python strings. So, if you want to search for the pattern *\t* in a string, you first need to precede ** with another ** for a literal match resulting in the string *\\t*. But when you are passing this as an argument to *re.compile( )* as a string, you have to precede each of these ** with yet another ** because Python strings also consider ** as a special symbol. Thus, the simply insane regular expression *\\\\t* only will result in a match for *\t*. In order to overcome this problem, Python regular expressions use the raw string notation which keeps the regular expressions simple. In raw string notation, every regular expression string is prefixed with an *r*